

Лабораторная работа № 3

Daisyworld: агентная модель саморегуляции

Куокконен Дарина Андреевна

2026-08-31

Аннотация

Исследована пространственная агентная модель Daisyworld. Чёрные и белые маргаритки изменяют температуру поверхности через альбедо, а температура определяет вероятность их размножения. Реализованы начальные состояния, анимация, временные ряды численности, сценарий изменения солнечной светимости и три параметрических расширения. Сопоставлены четыре сочетания возраста растений и начальной доли белых маргариток. Все сценарии представлены самостоятельными источниками Julia, таблицами, графиками и блокнотами.

Российский университет дружбы народов имени Патриса Лумумбы.
Учебная группа НФИбд-01-23.

Содержание

1. Цель и задание
2. Математическая и агентная модель
3. Вычислительный проект и реализация
4. Основные эксперименты
5. Параметрические исследования
6. Литературное программирование и Jupyter
7. Проверки и воспроизведение
8. Выводы
9. Источники

1. Цель и задание

Цель работы — исследовать обратную связь между популяцией и средой в пространственной модели Daisyworld средствами Agents.jl. Содержание соответствует главе 3 практикума А. В. Корольковой и Д. С. Кулябова, стр. 115–135.

Необходимо выполнить семь самостоятельных сценариев: построить состояния поля, получить анимацию, проследить численности при постоянной светимости, исследовать изменяющуюся светимость и повторить три вычислительных сценария для сетки параметров. Отдельно подготовлены производные форматы Literate и их включение в отчёт. Генерируемые копии сценария не считаются новым экспериментом.

Таблица 1: Самостоятельные сценарии лабораторной работы

Сценарий в scripts/	Назначение	Страницы
daisyworld.jl	состояния при $t = 0, 5, 45$	123–124
daisyworld-animate.jl	60 кадров эволюции поля	124–125
daisyworld-count.jl	1000 шагов при постоянной светимости	125–127
daisyworld-luminosity.jl	1000 шагов, сценарий ramp	127–129
daisyworld__param.jl	четыре набора начальных условий	129–134
daisyworld-count__param.jl	четыре временных ряда численности	129–134
daisyworld-luminosity__param.jl	четыре опыта со светимостью	129–134

2. Математическая и агентная модель

2.1 Пространство, агенты и начальное состояние

Мир представлен периодической решёткой 30×30 . Каждая из 900 ячеек может быть пустой или содержать одну маргаритку. Агент хранит координаты, вид, возраст и альбедо. Периодические границы означают, что у крайней ячейки есть соседи с противоположной стороны поля. При расчёте диффузии используется восемь соседей.

Таблица 2: Базовые параметры

Параметр	Базовое значение
Начальная доля белых и чёрных	0,2 и 0,2
Максимальный возраст	25 шагов
Альбедо белой / чёрной маргаритки	0,75 / 0,25
Альбедо пустой поверхности	0,4
Начальная светимость L	1,0
Seed генератора	165

В базовом начальном состоянии размещается по 180 растений каждого вида; остальные 540 ячеек свободны. Возраст начальных агентов случайный, поэтому исчезновение растений не синхронизировано одним моментом времени.

2.2 Нагрев и диффузия

Поглощённая доля энергии в ячейке равна

$$E_{ij} = (1 - a_{ij})L,$$

где a_{ij} — альбедо растения либо пустой поверхности. Локальная целевая температура задаётся формулой из методички:

$$T_{ij}^{loc} = 72 \ln E_{ij} + 80 (E_{ij} > 0).$$

При неположительном E_{ij} программный пример задаёт 80. В выполненных сценариях $L > 0$ и альбедо меньше единицы, поэтому эта защитная ветвь не используется. После вычисления локального значения температура усредняется с предыдущей:

$$T_{ij} \leftarrow (T_{ij} + T_{ij}^{loc}) / 2.$$

Диффузия переносит тепло к соседям:

$$T_{ij} \leftarrow 0,5 T_{ij} + \frac{0,5}{8} \sum_{(k,l) \in N_{ij}} T_{kl}.$$

Как в исходном коде практикума, обновление выполняется на месте в ходе обхода решётки. Это не синхронная схема с отдельным буфером температур: порядок обхода влияет на численные значения. Он зафиксирован в реализации.

2.3 Рождение, старение и обратная связь

Вероятность появления потомка зависит от температуры:

$$p(T) = \max(0, \min(1, 0,1457 T - 0,0032 T^2 - 0,6443))$$

Потомок наследует вид и альбедо, получает нулевой возраст и занимает случайную свободную соседнюю клетку. При отсутствии свободных соседей рождения нет. Возраст увеличивается при агентном шаге; достигшие `max_age` растения удаляются. Температура непосредственно влияет на размножение. Дополнительное правило гибели от холода или жары не вводилось, поскольку его нет в программном примере методички.

Чёрные растения поглощают больше энергии и согревают поверхность, белые отражают больше энергии и охлаждают её. Изменение температуры сказывается на размножении обоих видов. Саморегуляция возникает из этой обратной связи, а не из отдельной команды, удерживающей среднюю температуру постоянной.

2.4 Сценарий изменения светимости

Для `scenario = :ramp` после шагов 201–400 светимость увеличивается на 0,005 за шаг, после шагов 501–750 уменьшается на 0,0025. В остальные моменты она не изменяется. Поэтому L проходит уровни $1,0 \rightarrow 2,0 \rightarrow 1,375$. Значение в CSV записывается после модельного шага; нагрев этого шага использовал значение до последнего изменения светимости. Этот порядок сохранён из практикума.

3. Вычислительный проект и реализация

Проект содержит один модуль модели, общие функции построения результатов и семь литературных сценариев. `Project.toml` и `Manifest.toml` фиксируют зависимости. `Agents.jl` отвечает за решётку и планировщик, `CairoMakie` — за рисунки и анимацию, `CSV` и `DataFrames` — за таблицы, `DrWatson` — за пути.

```
cd /workspace/labs/lab03/project
julia --project=. -e 'using Pkg; Pkg.instantiate()'
make test
```

```
dakuokkonen@lab03:/workspace/labs/lab03/project$ julia --version
julia version 1.11.9
dakuokkonen@lab03:/workspace/labs/lab03/project$ quarto --version
1.10.18
dakuokkonen@lab03:/workspace/labs/lab03/project$ ls scripts/*.jl
scripts/daisyworld-animate.jl      scripts/daisyworld-luminosity.jl      scripts/daisyworld_param.jl
scripts/daisyworld-count.jl        scripts/daisyworld-luminosity_param.jl  scripts/setup_environment.jl
scripts/daisyworld-count_param.jl  scripts/daisyworld.jl                  scripts/tangle.jl
dakuokkonen@lab03:/workspace/labs/lab03/project$
```

Рисунок 1: Версии инструментов и состав самостоятельных сценариев

```
dakuokkonen@lab03:/workspace/labs/lab03/project$ sed -n '1,28p' src/daisyworld.jl
module DaisyworldModel

using Agents, Random, Statistics
using StatsBase: sample

export Daisy,
    daisyworld,
    daisy_step!,
    daisyworld_step!,
    measure,
    advance!,
    update_surface_temperature!,
    diffuse_temperature!,
    propagate!,
    solar_activity!

@agent struct Daisy(GridAgent{2})
    breed::Symbol
    age::Int
    albedo::Float64
end

function update_surface_temperature!(pos, model)
    albedo =
        isempty(pos, model) ? model.surface_albedo :
        model[id_in_position(pos, model)].albedo
    absorbed = (1 - albedo) * model.solar_luminosity
    heating = absorbed > 0 ? 72 * log(absorbed) + 80 : 80.0
end
```

Рисунок 2: Открытие общего модуля модели в терминале

3.1 Общий модуль

Ниже приведён полный листинг `src/daisyworld.jl`. Инициализация проверяет доли, возраст и размер поля; все случайные операции используют генератор самой модели. Функция `measure` собирает состояние после шага в одну строку.

В примере практикума часть случайных выборов не связана с генератором модели. Здесь им явно передаётся `abm rng(model)`: это техническое исправление сохраняет правила переходов, но делает одинаковый `seed` достаточным для повторения опыта.

```
module DaisyworldModel
```

```

using Agents, Random, Statistics
using StatsBase: sample

export Daisy,
    daisyworld,
    daisy_step!,
    daisyworld_step!,
    measure,
    advance!,
    update_surface_temperature!,
    diffuse_temperature!,
    propagate!,
    solar_activity!

@agent struct Daisy(GridAgent{2})
    breed::Symbol
    age::Int
    albedo::Float64
end

function update_surface_temperature!(pos, model)
    albedo =
        isempty(pos, model) ? model.surface_albedo :
        model[id_in_position(pos, model)].albedo
    absorbed = (1 - albedo) * model.solar_luminosity
    heating = absorbed > 0 ? 72 * log(absorbed) + 80 : 80.0
    model.temperature[pos...] =
        (model.temperature[pos...] + heating) / 2
end

function diffuse_temperature!(pos, model)
    neighbors = nearby_positions(pos, model)
    model.temperature[pos...] =
        (1 - model.ratio) * model.temperature[pos...] +
        model.ratio * sum(model.temperature[p...] for p in neighbors) /
        8
end

function propagate!(pos, model)
    isempty(pos, model) && return
    parent = model[id_in_position(pos, model)]
    temperature = model.temperature[pos...]
    probability = clamp(
        0.1457 * temperature - 0.0032 * temperature^2 - 0.6443,
        0,
        1,
    )
    if rand(abrng(model)) < probability
        target = random_nearby_position(
            pos,
            model,
            1,
            p -> isempty(p, model),
        )
        isnothing(target) ||
            add_agent!(target, model, parent.breed, 0, parent.albedo)
    end
    nothing
end

function daisy_step!(agent, model)
    agent.age += 1
    agent.age >= model.max_age && remove_agent!(agent, model)
    nothing
end

```

```

end

function solar_activity!(model)
    if model.scenario == :ramp
        200 < model.tick <= 400 &&
            (model.solar_luminosity += model.solar_change)
        500 < model.tick <= 750 &&
            (model.solar_luminosity -= model.solar_change / 2)
    elseif model.scenario == :change
        model.solar_luminosity += model.solar_change
    end
    nothing
end

function daisyworld_step!(model)
    for pos in positions(model)
        update_surface_temperature!(pos, model)
        diffuse_temperature!(pos, model)
        propagate!(pos, model)
    end
    model.tick += 1
    solar_activity!(model)
end

function daisyworld(;
    griddims = (30, 30),
    max_age = 25,
    init_white = 0.2,
    init_black = 0.2,
    albedo_white = 0.75,
    albedo_black = 0.25,
    surface_albedo = 0.4,
    solar_change = 0.005,
    solar_luminosity = 1.0,
    scenario = :default,
    seed = 165,
)
    all(>=(3), griddims) ||
        throw(ArgumentError("grid dimensions must be at least 3"))
    max_age > 0 || throw(ArgumentError("max_age must be positive"))
    0 <= init_white <= 1 &&
    0 <= init_black <= 1 &&
    init_white + init_black <= 1 || throw(
        ArgumentError("initial fractions must sum to at most one"),
    )
    all(
        x -> 0 <= x <= 1,
        (albedo_white, albedo_black, surface_albedo),
    ) || throw(ArgumentError("albedo outside [0,1]"))
    scenario in (:default, :ramp, :change) ||
        throw(ArgumentError("unknown scenario"))
    properties = Dict{Symbol,Any}(
        :max_age=>max_age,
        :surface_albedo=>surface_albedo,
        :solar_luminosity=>solar_luminosity,
        :solar_change=>solar_change,
        :scenario=>scenario,
        :tick=>0,
        :ratio=>0.5,
        :temperature=>zeros(griddims),
    )
    model = StandardABM(
        Daisy,
        GridSpaceSingle(griddims; periodic = true);

```

```

        properties,
        rng = MersenneTwister(seed),
        agent_step! = daisy_step!,
        model_step! = daisyworld_step!,
        scheduler = Schedulers.fastest,
    )
    cells = collect(positions(model))
    white_cells = sample(
        abmrng(model),
        cells,
        floor{Int, init_white * length(cells)};
        replace = false,
    )
    for pos in white_cells
        add_agent!(
            pos,
            model,
            :white,
            rand(abmrng(model), 0:max_age),
            albedo_white,
        )
    end
    black_cells = sample(
        abmrng(model),
        setdiff(cells, white_cells),
        floor{Int, init_black * length(cells)};
        replace = false,
    )
    for pos in black_cells
        add_agent!(
            pos,
            model,
            :black,
            rand(abmrng(model), 0:max_age),
            albedo_black,
        )
    end
    for pos in positions(model)
        update_surface_temperature!(pos, model)
    end
    model
end

advance!(model, n = 1) = Agents.step!(model, n)

function measure(model)
    black = count(a -> a.breed == :black, allagents(model))
    white = count(a -> a.breed == :white, allagents(model))
    albedo = mean(
        isempty(p, model) ? model.surface_albedo :
        model[id_in_position(p, model)].albedo for
        p in positions(model)
    )
    (;
        time = model.tick,
        black,
        white,
        temperature = mean(model.temperature),
        luminosity = model.solar_luminosity,
        albedo,
    )
end

end

```


3.2 Сохранение наблюдений и графиков

В `src/experiments.jl` находятся общие функции трёх типов: снимок решётки, сбор временного ряда и построение панелей. В CSV включается начальная точка, поэтому опыт на 1000 шагов содержит 1001 строку. Среднее альbedo вычисляется по всей поверхности, включая свободные ячейки, а не только по растениям.

```
using Agents, CSV, DataFrames, DrWatson, Statistics, CairoMakie
include(joinpath(@__DIR__, "daisyworld.jl"))
using .DaisyworldModel

imagefile(name) =
    (mkpath(projectdir("../", "image")); projectdir("../", "image", name))
datafile(name, file) = (mkpath(datadir(name)); datadir(name, file))
set_theme!(Theme(font = "DejaVu Sans", fontsize = 16))

function snapshot(model; title = "Daisyworld")
    fig = Figure(size = (850, 700))
    ax = Axis(
        fig[1, 1],
        title = "$title · t=$(model.tick)",
        xlabel = "x",
        ylabel = "y",
        aspect = 1,
    )
    hm = heatmap!(
        ax,
        model.temperature;
        colormap = :thermal,
        colorrange = (-20, 60),
    )
    for (breed, color) in ([:white, :white], [:black, :black])
        plants = [a.pos for a in allagents(model) if a.breed == breed]
        isempty(plants) || scatter!(
            ax,
            [p[1] for p in plants],
            [p[2] for p in plants];
            color,
            marker = :circle,
            markersize = 8,
            strokewidth = 0.4,
            strokecolor = :gray,
        )
    end
    Colorbar(fig[1, 2], hm; label = "Температура, °C")
    fig
end

function history(model, steps = 1000)
    rows = [measure(model)]
    for _ in 1:steps
        advance!(model)
        push!(rows, measure(model))
    end
    DataFrame(rows)
end

function countfigure(df; title = "Численность маргариток")
    fig = Figure(size = (1000, 550))
    ax = Axis(
        fig[1, 1],
        title = title,
        xlabel = "Модельный шаг",
```

```

        ylabel = "Число растений",
    )
    lines!(ax, df.time, df.black; color = :black, label = "Чёрные")
    lines!(ax, df.time, df.white; color = :orange, label = "Белые")
    axislegend(ax; position = :rt)
    fig
end

function dynamicsfigure(df; title = "Daisyworld: обратная связь")
    fig = Figure(size = (1000, 1000))
    axes = [
        Axis(fig[i, 1], ylabel = label) for
        (i, label) in enumerate((
            "Число растений",
            "Температура, °C",
            "Светимость",
            "Среднее альбедо",
        ))
    ]
    axes[1].title = title
    lines!(axes[1], df.time, df.black; color = :black, label = "Чёрные")
    lines!(axes[1], df.time, df.white; color = :orange, label = "Белые")
    axislegend(axes[1]; position = :rt)
    for (ax, column) in
        zip(axes[2:4], (:temperature, :luminosity, :albedo))
    lines!(ax, df.time, df[:, column]; color = :royalblue)
    end
    axes[4].xlabel = "Модельный шаг"
    linkxaxes!(axes...)
    fig
end

parameter_sets() = dict_list(
    Dict(
        :max_age=>[25, 40],
        :init_white=>[0.2, 0.8],
        :init_black=>0.2,
        :seed=>165,
    ),
)

function savehistory(df, name)
    CSV.write(datafile(name, "trajectory.csv"), df)
    show(last(df, 5); allcols = true)
    println()
    df
end

```

4. Основные эксперименты

4.1 Начальное состояние и развитие пространственной структуры

Сценарий выполняет пять шагов, затем ещё сорок. Последний снимок соответствует моменту 45, а не 40: это исправление подписи, не изменение числа выполненных шагов. Литературный фрагмент ниже автоматически получен из `daisyworld.jl`.

4.1.1 Daisyworld: пространственное состояние

Базовый опыт: сетка 30×30, два вида, светимость 1.0, seed 165.

4.1.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(scrdir("experiments.jl"))
```

4.1.1.2 Последовательная эволюция

```
model = daisyworld()
snapshots = DataFrame[]
for (i, target) in enumerate((0, 5, 45))
    advance!(model, target - model.tick)
    push!(snapshots, DataFrame([measure(model)]))
    figure = snapshot(model)
    save(imagefile(lpad(i, 2, '0') * "-plot.png"), figure)
    display(figure)
end
```

4.1.1.3 Сохранённые измерения

```
summary = reduce(vcat, snapshots)
CSV.write(datafile("daisyworld", "snapshots.csv"), summary)
summary
```

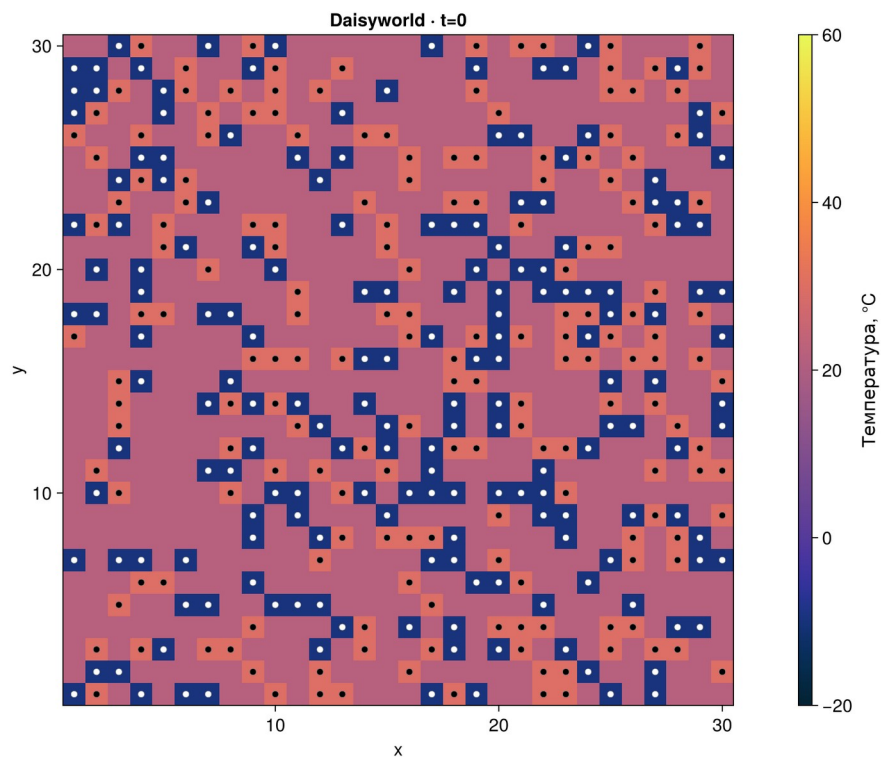


Рисунок 3: Начальное размещение маргариток, $t = 0$

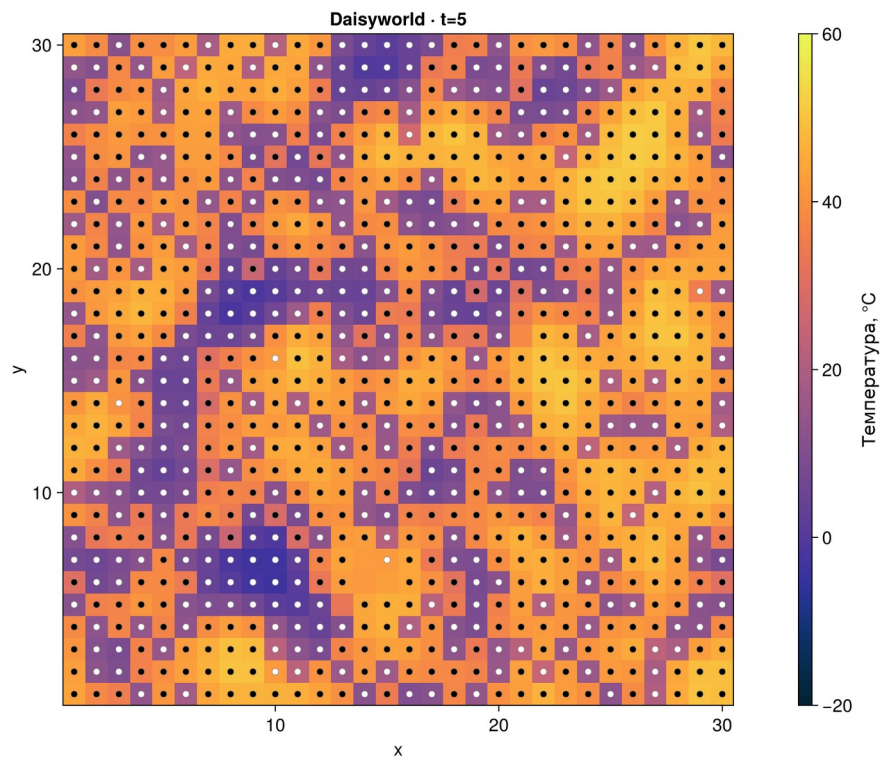


Рисунок 4: Состояние поля после пяти шагов

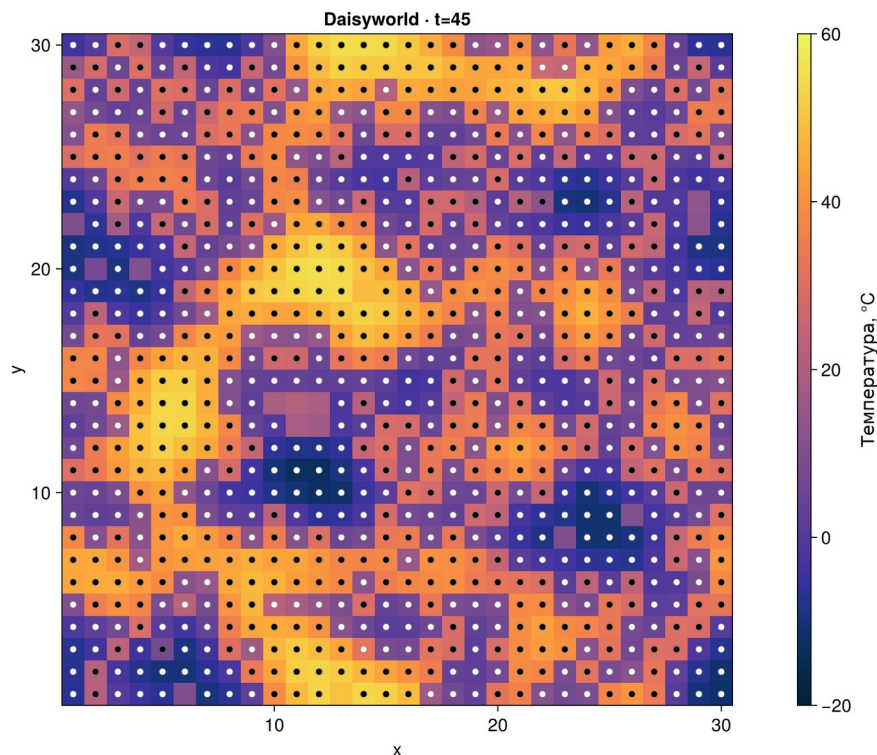


Рисунок 5: Состояние после суммарных 45 шагов

Таблица 3: Контрольные состояния базового опыта

Шаг	Чёрные	Белые	Средняя Т, °C	Альбе́до
0	180	180	16,914	0,44
5	563	332	30,365	0,435
45	450	435	20,129	0,494

Начальное состояние содержит свободные клетки и смешанное размещение видов (Рисунок 3). Сравнение (Рисунок 4; Рисунок 5) позволяет проследить формирование участков с различной температурой. Цветовое поле и отметки растений описывают разные переменные: тёмный фон сам по себе не означает наличие чёрной маргаритки.

4.2 Анимация

Отдельный сценарий записывает 60 последовательных состояний $t = 0 \dots 59$ с частотой 10 кадров в секунду. Результат `image/01-animation.gif` показывает эволюцию одной модели, а таблица `data/daisyworld-animate/frames.csv` связывает кадры с фактическим временем и численностями. GIF остаётся динамическим приложением; печатный отчёт содержит контрольные состояния из предыдущего опыта.

4.2.1 Daisyworld: анимация

Сохраним 60 последовательных состояний модели в анимации GIF.

4.2.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(scrdir("experiments.jl"))
```

4.2.1.2 Наблюдаемые величины

```
model = daisyworld()
fig = Figure(size = (850, 700))
ax = Axis(
    fig[1, 1],
    title = "Daisyworld",
    xlabel = "x",
    ylabel = "y",
    aspect = 1,
)
temperature = Observable(copy(model.temperature))
black_positions = Observable(Point2f[])
white_positions = Observable(Point2f[])
hm = heatmap!(
    ax,
    temperature;
    colormap = :thermal,
    colrange = (-20, 60),
)
scatter!(ax, black_positions; color = :black, markersize = 8)
scatter!(ax, white_positions; color = :white, markersize = 8)
Colorbar(fig[1, 2], hm; label = "Температура, °C")
```

4.2.1.3 60 состояний от $t = 0$ до $t = 59$

```
rows = NamedTuple[]
record(
    fig,
    imagefile("01-animation.gif"),
    0:59;
    framerate = 10,
) do frame
    frame > 0 && advance!(model)
    temperature[] = copy(model.temperature)
    black_positions[] = [
        Point2f(a.pos...) for a in allagents(model) if a.breed == :black
    ]
    white_positions[] = [
        Point2f(a.pos...) for a in allagents(model) if a.breed == :white
    ]
    ax.title = "Daisyworld · t=$(model.tick)"
    push!(rows, measure(model))
end
```

4.2.1.4 Данные и последний кадр

```
CSV.write(datafile("daisyworld-animate", "frames.csv"), DataFrame(rows))
display(fig)
DataFrame(rows)[1:10:end, :]
```

4.3 Численность при постоянной светимости

В `daisyworld-count.jl` исходная модель запускается заново с теми же параметрами и `seed`. На 1000 шагах регистрируются оба вида; светимость остаётся равной единице. Это отдельный опыт, а не продолжение состояния $t = 45$.

4.3.1 Daisyworld: численность

Проследим численность чёрных и белых растений на 1000 шагах.

4.3.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(scrdir("experiments.jl"))
```

4.3.1.2 Расчёт

```
model = daisyworld(; solar_luminosity = 1.0)
df = savehistory(history(model, 1000), "daisyworld-count")
```

4.3.1.3 График и вывод

```
figure = countfigure(df)
save(imagefile("04-plot.png"), figure)
display(figure)
last(df, 5)
```

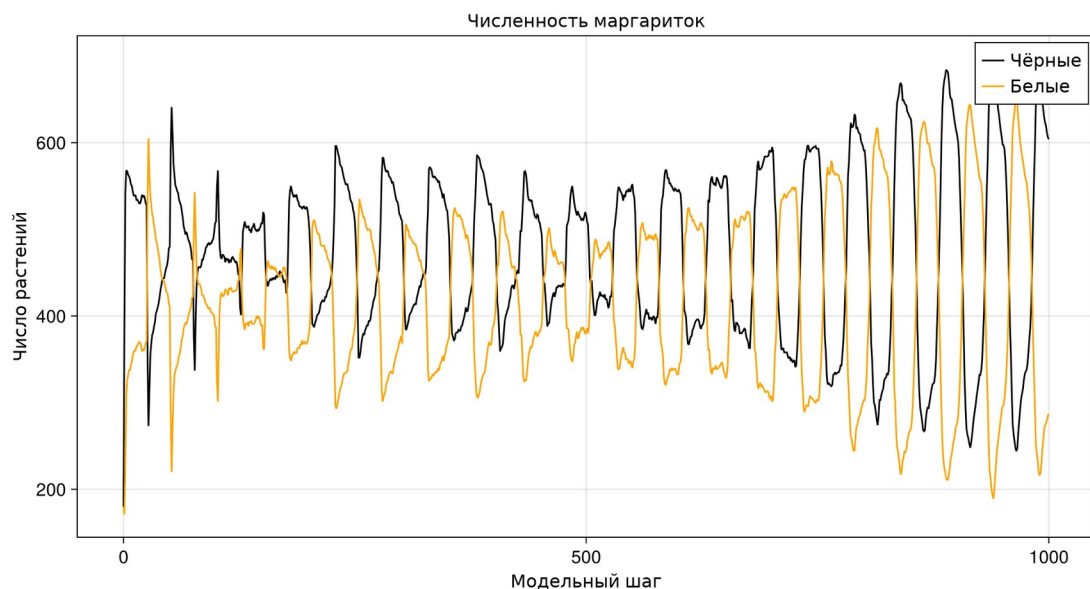


Рисунок 6: Численности при постоянной светимости

К шагу 1000 сохраняются оба вида: 604 чёрных и 287 белых растений. За весь опыт численность чёрных изменяется от 180 до 697, белых — от 171 до 653. Временной ряд содержит колебания, а не монотонный выход к одной постоянной численности. Средняя температура в конце равна 34,754 °C; это значение отдельного момента, а не среднее за весь опыт.

4.4 Реакция на изменение солнечной светимости

Четыре панели (Рисунок 7) показывают численности, среднюю температуру, светимость и среднее альbedo. Панель альbedo добавлена согласно текстовому заданию практикума; в демонстрационном фрагменте кода она отсутствует.

4.4.1 Daisyworld: изменение светимости

Сценарий ramp: рост на шагах 201–400, снижение на 501–750.

4.4.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(scrdir("experiments.jl"))
```

4.4.1.2 Расчёт четырёх связанных величин

```
model = daisyworld(; solar_luminosity = 1.0, scenario = :ramp)
df = savehistory(history(model, 1000), "daisyworld-luminosity")
```

4.4.1.3 Численность, температура, светимость и альbedo

```
figure = dynamicsfigure(df)
save(imagefile("05-plot.png"), figure)
display(figure)
df[[1, 201, 401, 501, 751, 1001], :]
```

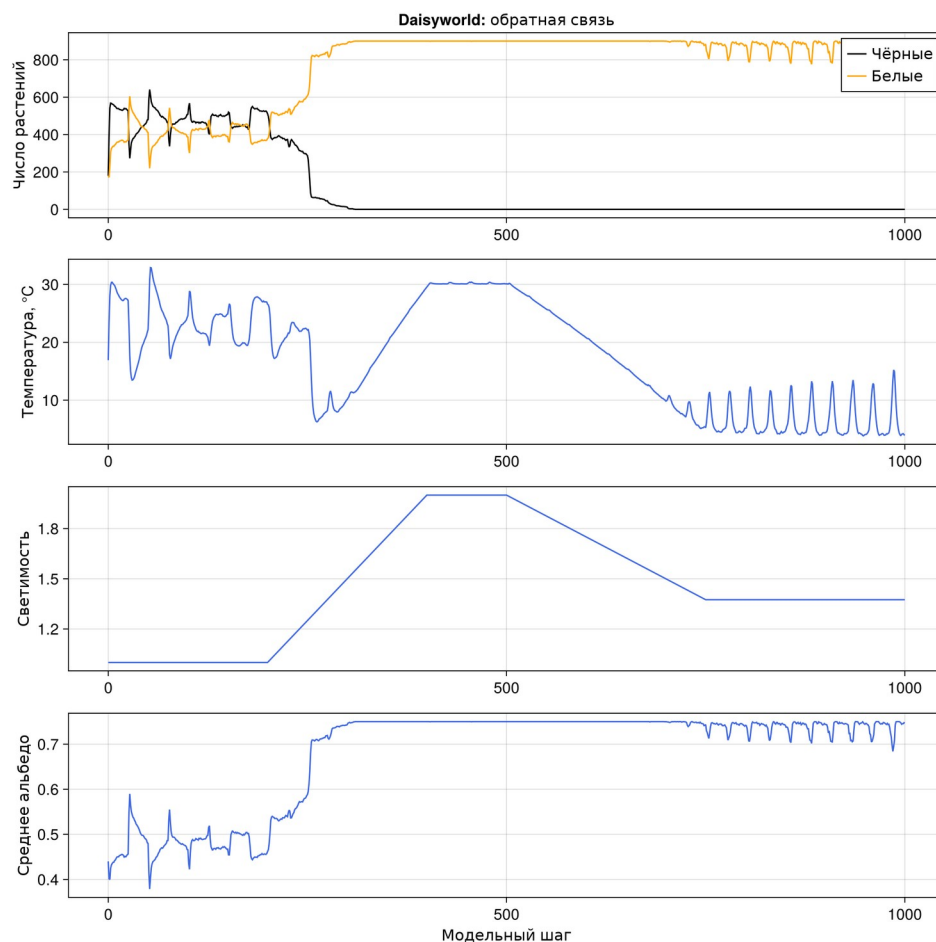


Рисунок 7: Численности и состояние среды в сценарии ramp

Таблица 4: Контрольные точки изменения светимости

Шаг	L	Чёрные	Белые	T, °C	Альбе́до
0	1	180	180	16,914	0,44
200	1	512	378	26,589	0,462
400	2	0	900	29,556	0,75
500	2	0	900	30,105	0,75
750	1,375	0	883	5,247	0,743
1 000	1,375	0	892	3,938	0,747

В данной траектории чёрные маргаритки исчезают на шаге 310, когда $L = 1,55$. В конце остаётся 892 белых растений, среднее альбе́до равно 0,747, температура — 3,938 °C. Увеличение светимости сначала меняет состав популяции в пользу белых растений. После исчезновения чёрного вида обратное уменьшение L не восстанавливает исходный состав: внешнего заселения и мутаций в модели нет. Следовательно, реакция зависит от предыстории, а саморегуляция не означает сохранения обоих видов при любом воздействии.

5. Параметрические исследования

5.1 План эксперимента

DrWatson `dict_list` формирует декартово произведение $\text{max_age} \in \{25, 40\}$ и $\text{init_white} \in \{0.2, 0.8\}$. Начальная доля чёрных фиксирована на 0,2, `seed` — 165. При $\text{init_white} = 0.8$ поле первоначально заполнено полностью; появление потомства возможно лишь после освобождения клеток. На каждое сочетание приходится самостоятельный запуск, а не повторное использование одной модели.

5.2 Параметрические состояния

5.2.1 Daisyworld: параметрические состояния

Возраст 25/40 и начальная доля белых 0.2/0.8: четыре комбинации.

5.2.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(scrdir("experiments.jl"))
```

5.2.1.2 Все комбинации, одинаковый seed

```
rows = NamedTuple[]
for (i, params) in enumerate(parameter_sets())
    model = daisyworld(; params...)
    for (j, target) in enumerate((0, 5, 45))
        advance!(model, target - model.tick)
        push!(
            rows,
            merge(
                (
                    max_age = params[:max_age],
                    init_white = params[:init_white],
                ),
            ),
        )
    end
end
```

```

        measure(model),
    ),
)
figure = snapshot(
    model;
    title = "Возраст $(params[:max_age]), белые $(
(params[:init_white])",
)
save(
    imagefile(lpad(5 + 3*(i-1) + j, 2, '0') * "-plot.png"),
    figure,
)
display(figure)
end
end
end

```

5.2.1.3 Сравнение состояний

```

summary = DataFrame(rows)
CSV.write(datafile("daisyworld__param", "snapshots.csv"), summary)
summary

```

5.2.2 Максимальный возраст 25, начальная доля белых 0,2

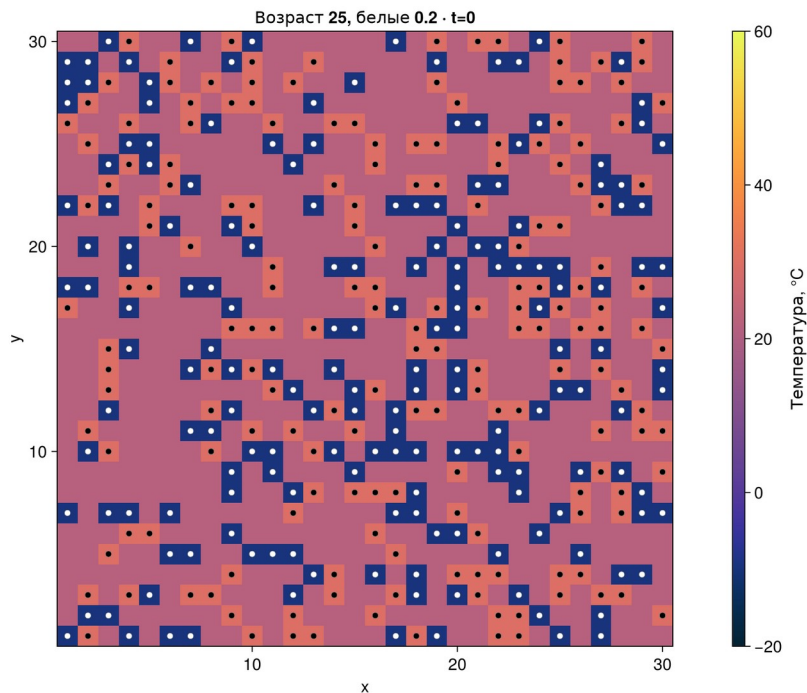


Рисунок 8: Параметрический опыт 1, $t = 0$

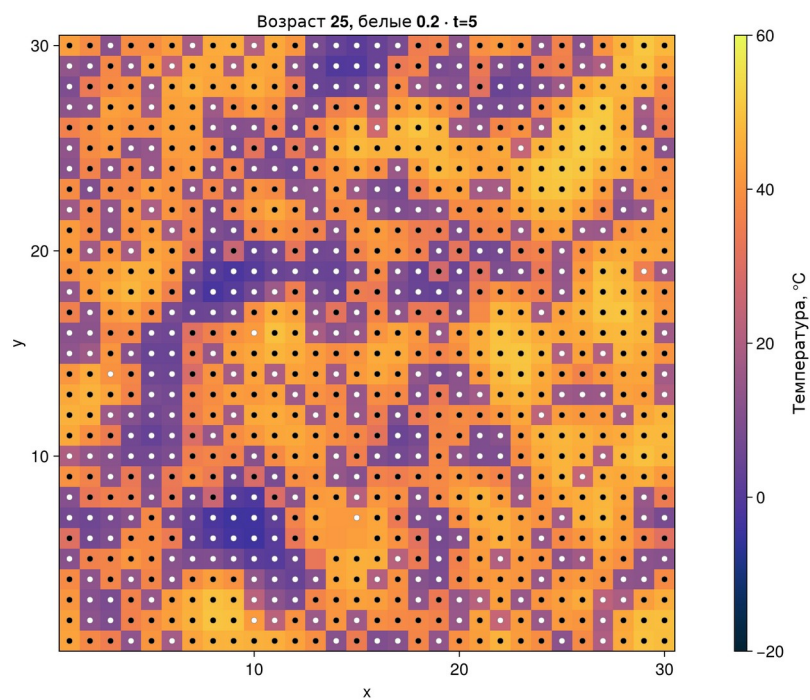


Рисунок 9: Параметрический опыт 1, $t = 5$

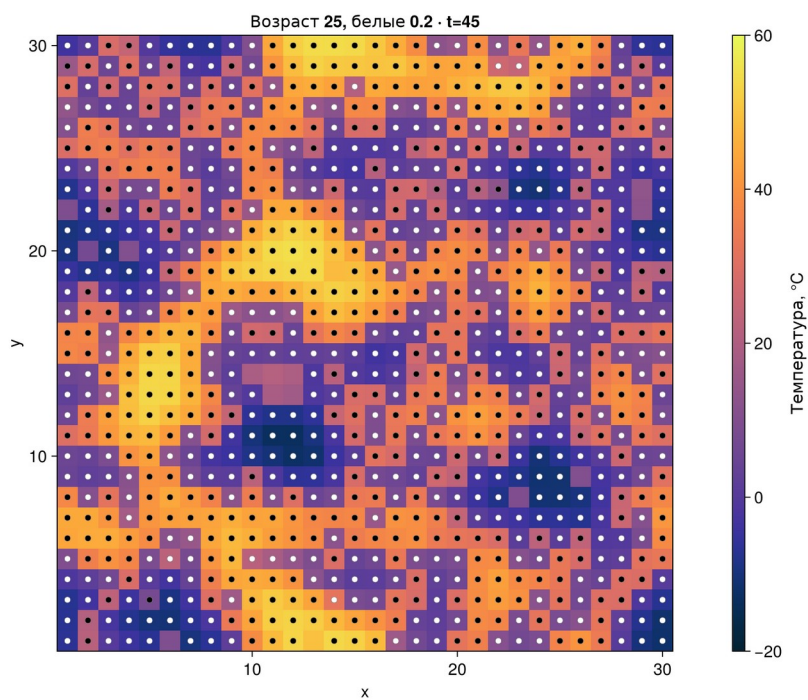


Рисунок 10: Параметрический опыт 1, $t = 45$

5.2.3 Максимальный возраст 25, начальная доля белых 0,8

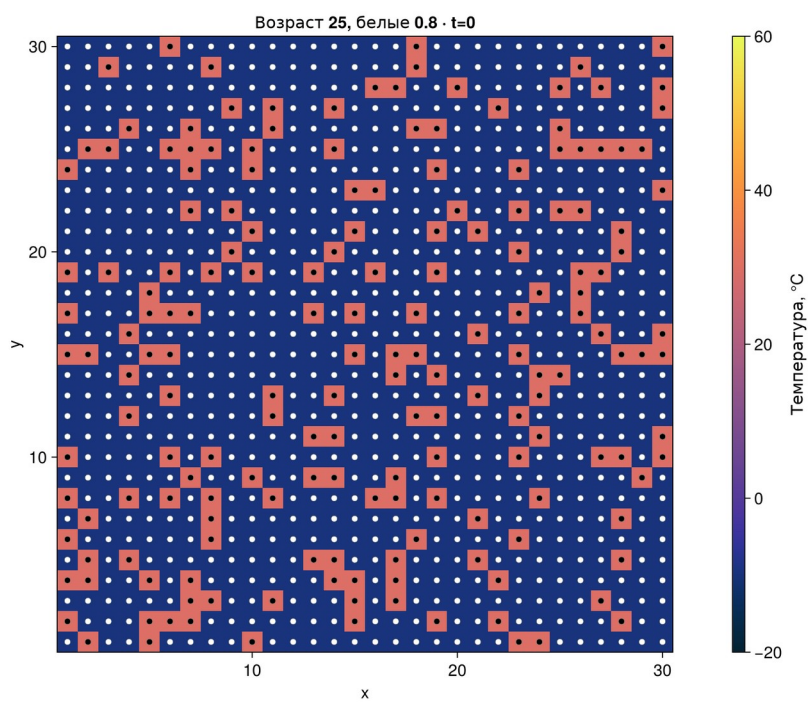


Рисунок 11: Параметрический опыт 2, $t = 0$

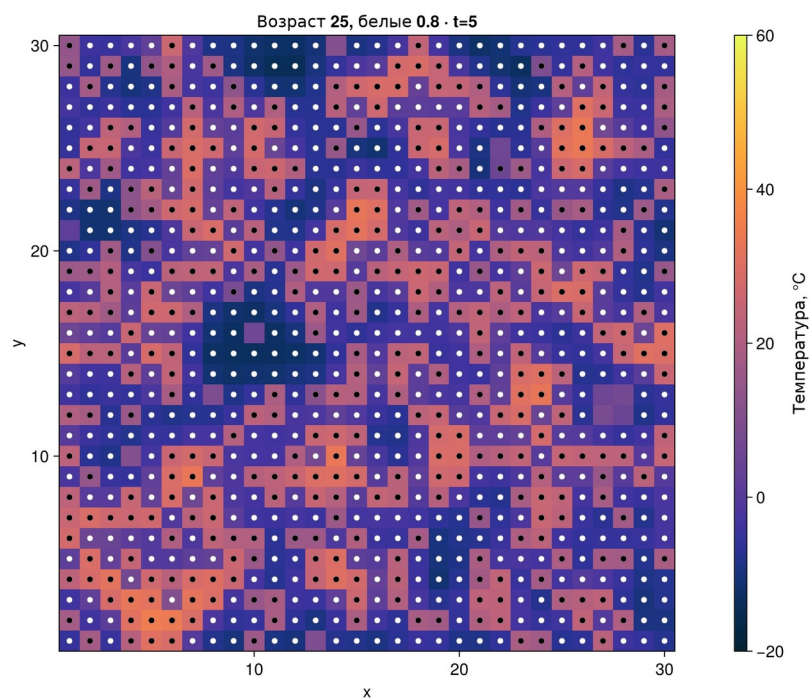


Рисунок 12: Параметрический опыт 2, $t = 5$

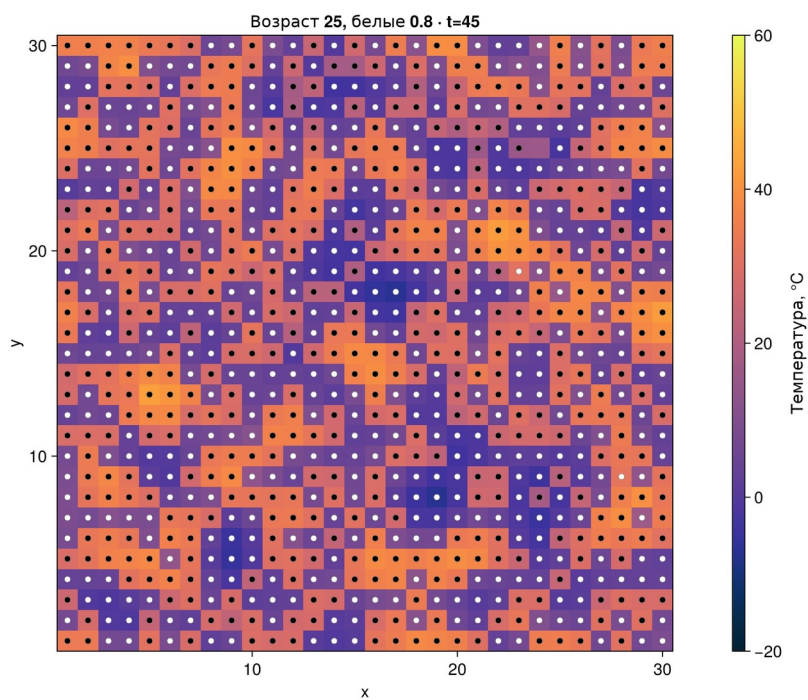


Рисунок 13: Параметрический опыт 2, $t = 45$

5.2.4 Максимальный возраст 40, начальная доля белых 0,2

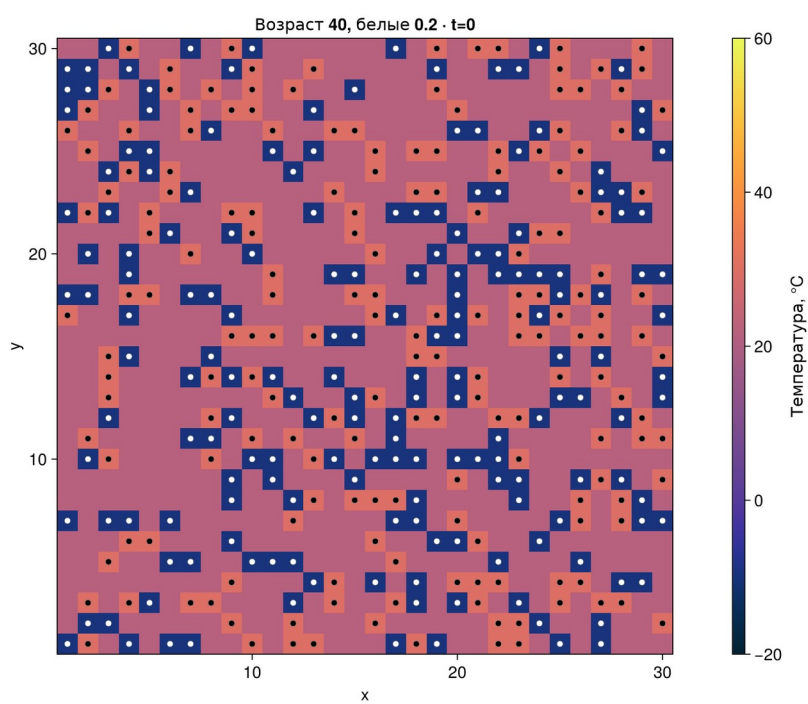


Рисунок 14: Параметрический опыт 3, $t = 0$

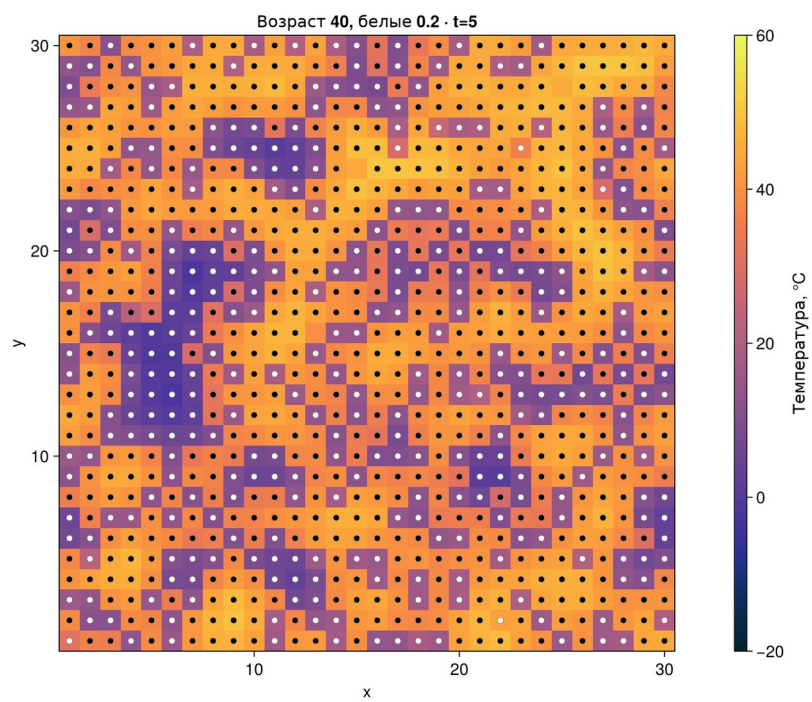


Рисунок 15: Параметрический опыт 3, $t = 5$

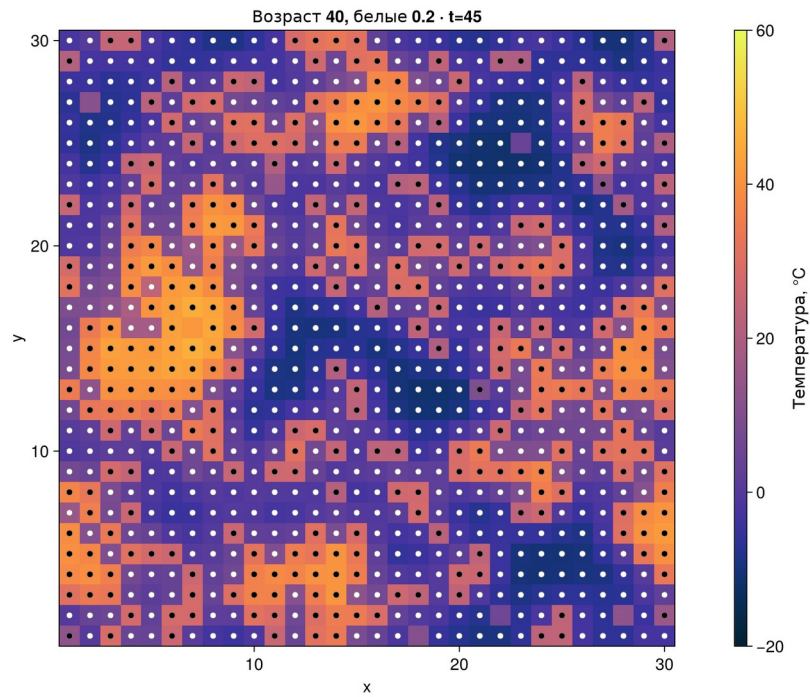


Рисунок 16: Параметрический опыт 3, $t = 45$

5.2.5 Максимальный возраст 40, начальная доля белых 0,8

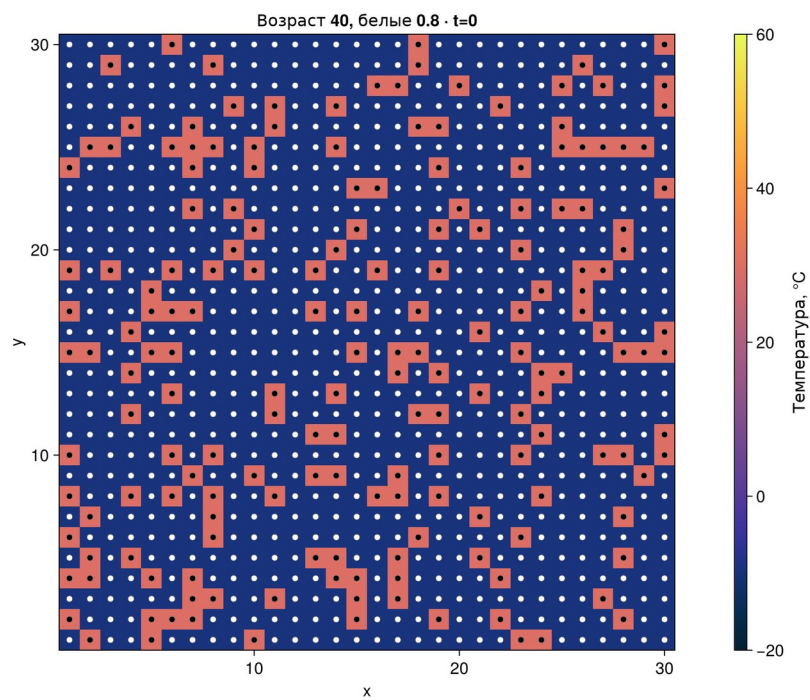


Рисунок 17: Параметрический опыт 4, $t = 0$

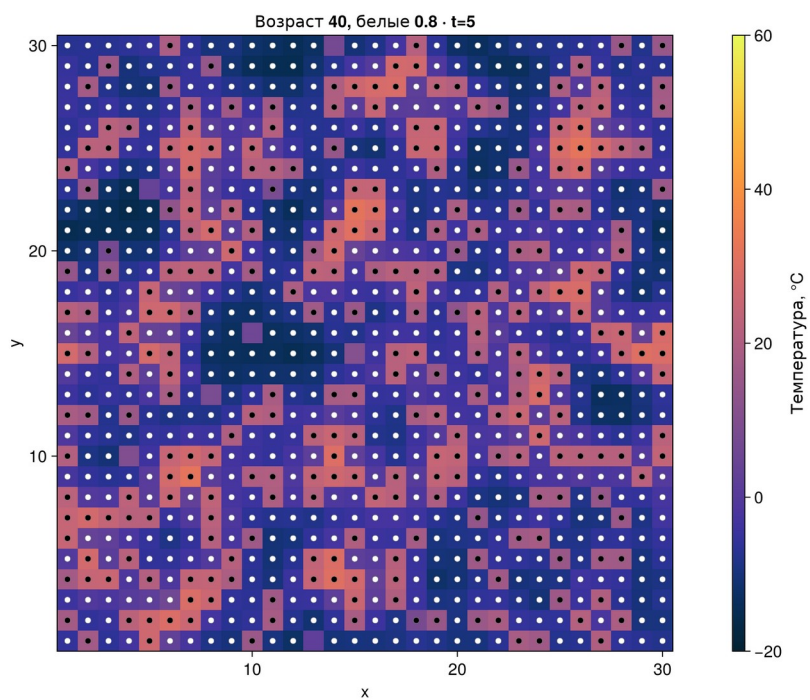


Рисунок 18: Параметрический опыт 4, $t = 5$

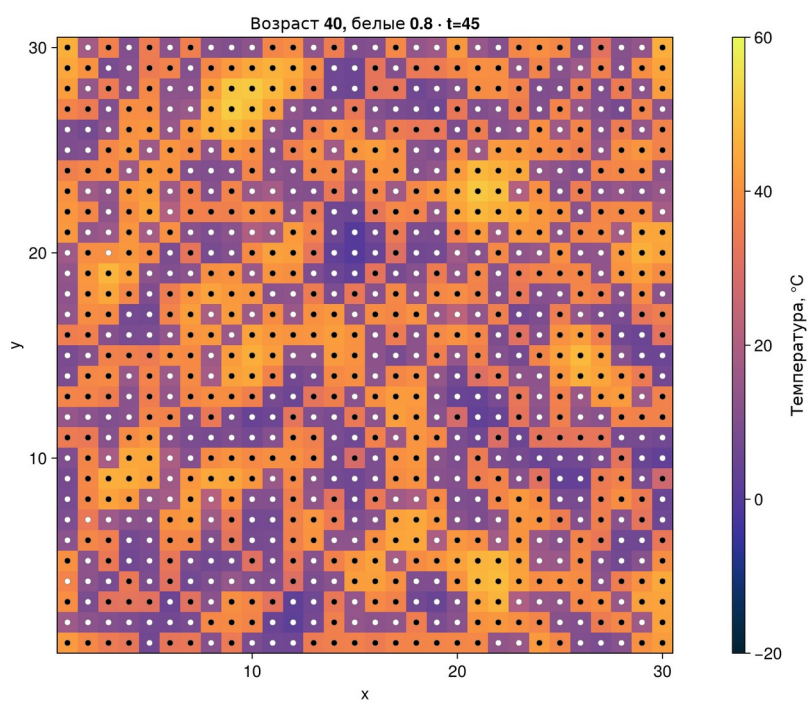


Рисунок 19: Параметрический опыт 4, $t = 45$

Таблица 5: Параметрические состояния на шаге 45

Возраст	Доля белых	Чёрные	Белые	T, °C
25	0,2	450	435	20,129
25	0,8	462	437	18,082
40	0,2	306	587	10,068
40	0,8	487	413	25,956

5.3 Численности при разных возрасте и составе

5.3.1 Daisyworld: параметры и численность

Четыре базовые комбинации, по 1000 шагов на каждую.

5.3.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(sourcedir("experiments.jl"))
```

5.3.1.2 Независимые прогоны

```
rows = NamedTuple[]
for (i, params) in enumerate(parameter_sets())
    model = daisyworld(; params...)
    df = history(model, 1000)
    CSV.write(
        datafile("daisyworld-count__param", "trajectory_$(i).csv"),
        df,
    )
    push!(
        rows,
        merge(
            (
                max_age = params[:max_age],
                init_white = params[:init_white],
            ),
            measure(model),
        ),
    )
    figure = countfigure(
        df;
        title = "Возраст $(params[:max_age]), белые $(params[:init_white])",
    )
    save(imagefile(lpad(17+i, 2, '0') * "-plot.png"), figure)
    display(figure)
end
```

5.3.1.3 Итоговые значения

```
summary = DataFrame(rows)
CSV.write(datafile("daisyworld-count__param", "summary.csv"), summary)
summary
```

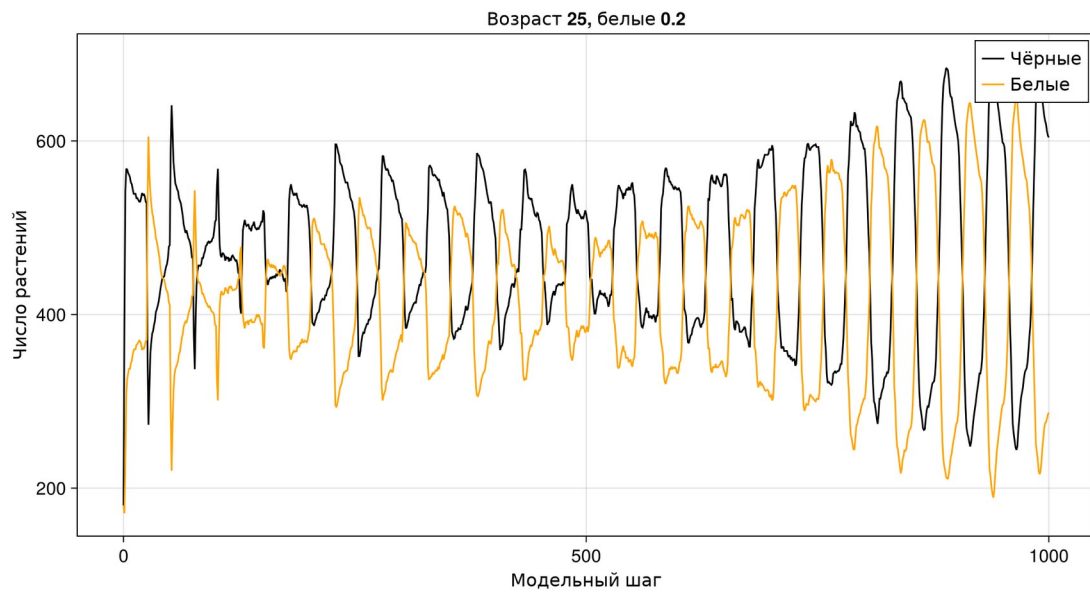


Рисунок 20: Численности: $\max_age = 25$, $\text{init_white} = 0.2$

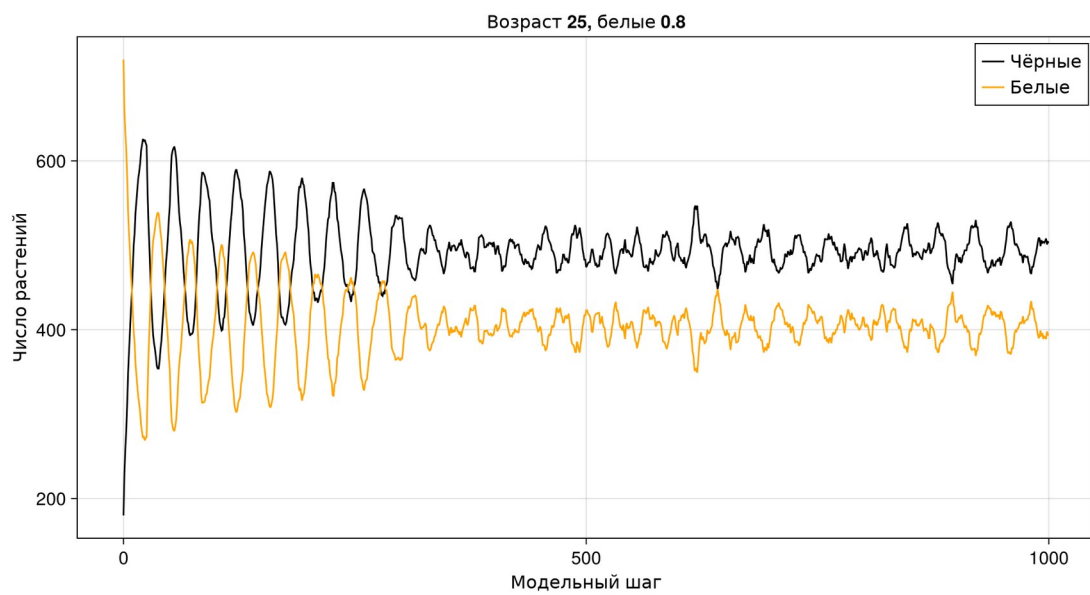


Рисунок 21: Численности: $\max_age = 25$, $\text{init_white} = 0.8$

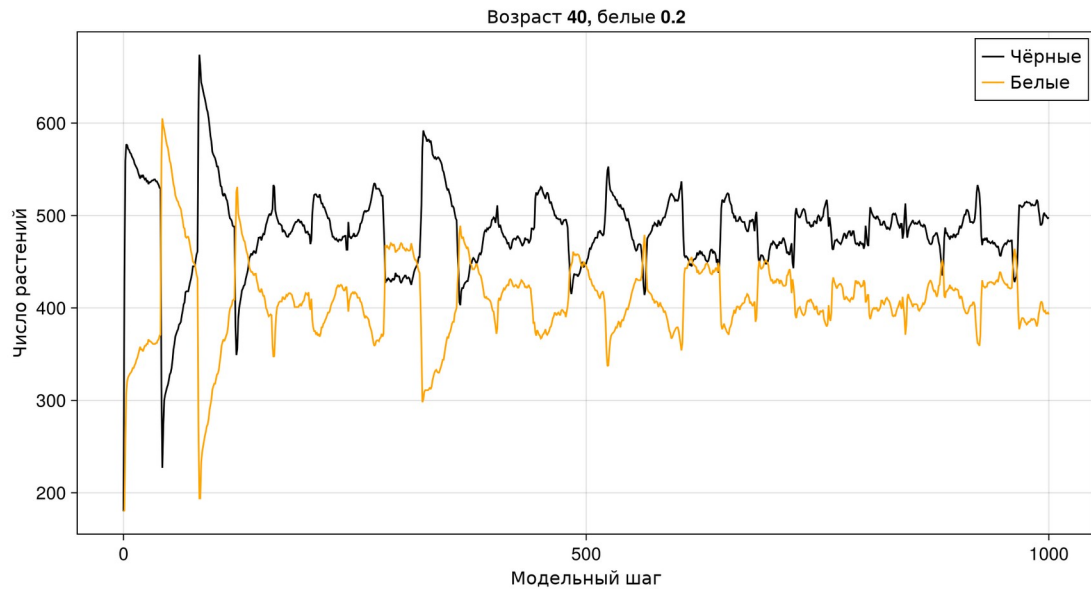


Рисунок 22: Численности: $\max_age = 40$, $\text{init_white} = 0.2$

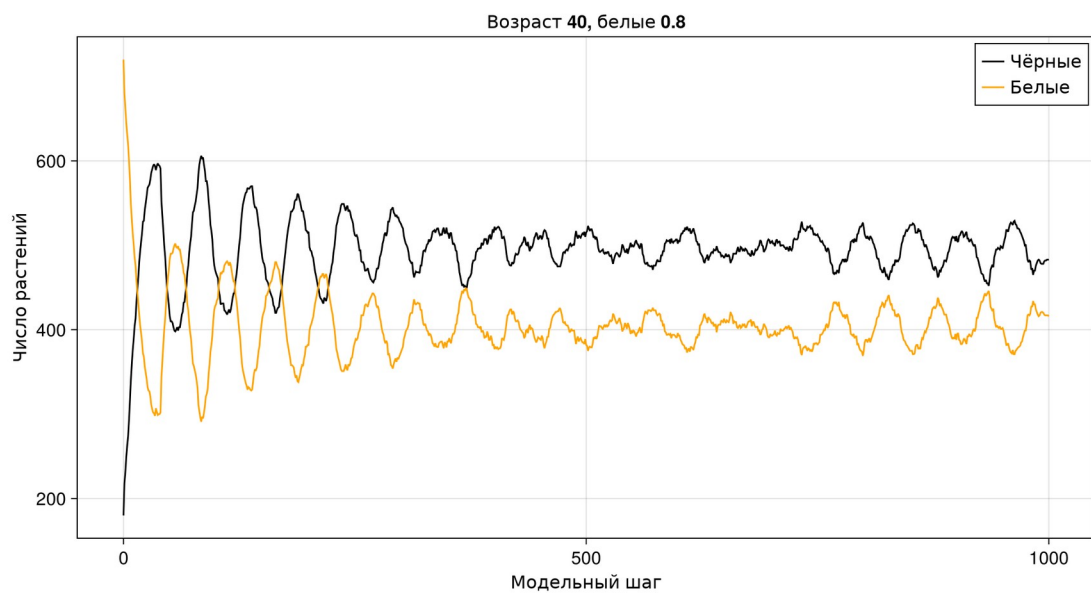


Рисунок 23: Численности: $\max_age = 40$, $\text{init_white} = 0.8$

Таблица 6: Конечные состояния при постоянной светимости

Возраст	Доля белых	Чёрные	Белые	T, °C
25	0,2	604	287	34,754
25	0,8	503	394	24,873
40	0,2	498	393	24,505
40	0,8	483	416	22,695

Во всех четырёх опытах при $L = 1$ оба вида сохраняются до шага 1000. Начальный состав и срок жизни меняют фазу и амплитуду колебаний. Сравнение только последней строки не задаёт строгого ранжирования устойчивости: популяции находятся в разных фазах динамики, поэтому таблицу нужно читать совместно с полными рядами.

5.4 Светимость и начальная популяция

5.4.1 Daisyworld: параметры при изменении светимости

Сопоставим устойчивость четырёх комбинаций в сценарии ramp.

5.4.1.1 Окружение и модель

```
using DrWatson
@quickactivate "project"
include(sourcedir("experiments.jl"))
```

5.4.1.2 Расчёт и полные временные ряды

```
rows = NamedTuple[]
for (i, params) in enumerate(parameter_sets())
    model = daisyworld(; params..., scenario = :ramp)
    df = history(model, 1000)
    CSV.write(
        datafile("daisyworld-luminosity__param", "trajectory_$(i).csv"),
        df,
    )
    push!(
        rows,
        merge(
            (
                max_age = params[:max_age],
                init_white = params[:init_white],
            ),
            measure(model),
        ),
    )
    figure = dynamicsfigure(
        df;
        title = "Возраст $(params[:max_age]), белые $(params[:init_white])",
    )
    save(imagefile(lpad(21+i, 2, '0') * "-plot.png"), figure)
    display(figure)
end
```

5.4.1.3 Сравнение конечных состояний

```
summary = DataFrame(rows)
CSV.write(
    datafile("daisyworld-luminosity__param", "summary.csv"),
    summary,
```

)
summary

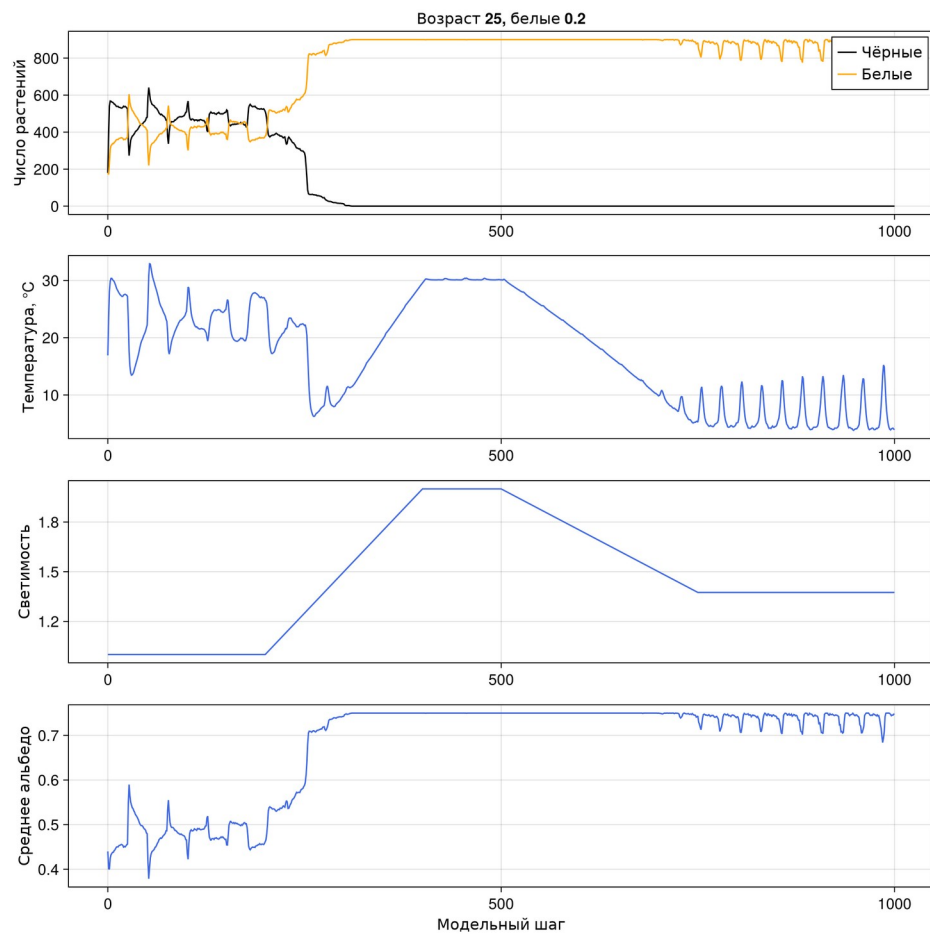


Рисунок 24: Сценарий ramp: $\text{max_age} = 25$, $\text{init_white} = 0.2$

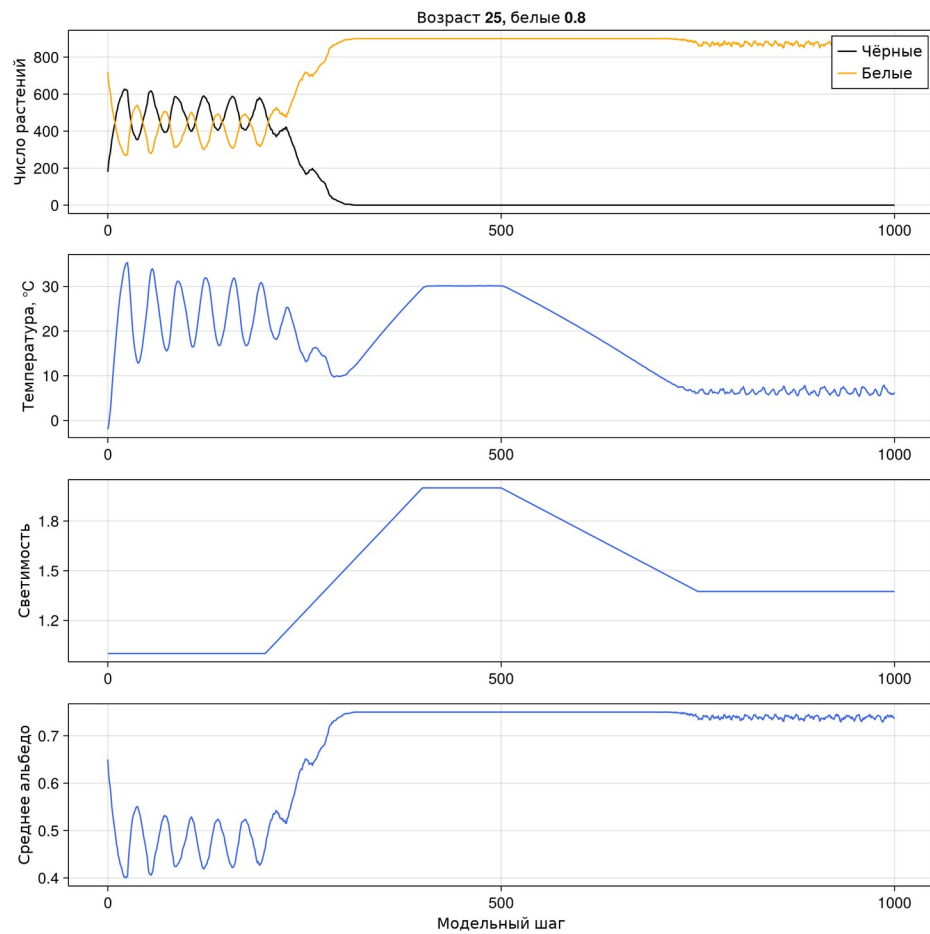


Рисунок 25: Сценарий ramp: $\max_age = 25$, $\text{init_white} = 0.8$

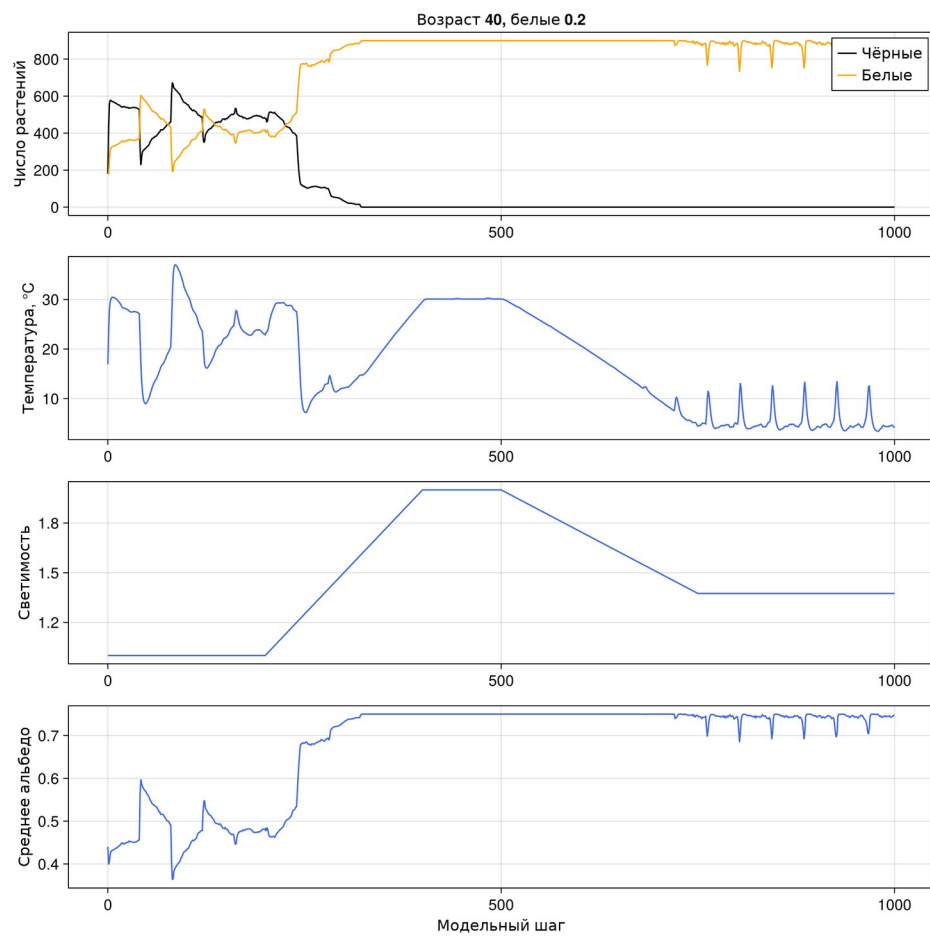


Рисунок 26: Сценарий ramp: $\max_age = 40$, $\text{init_white} = 0.2$

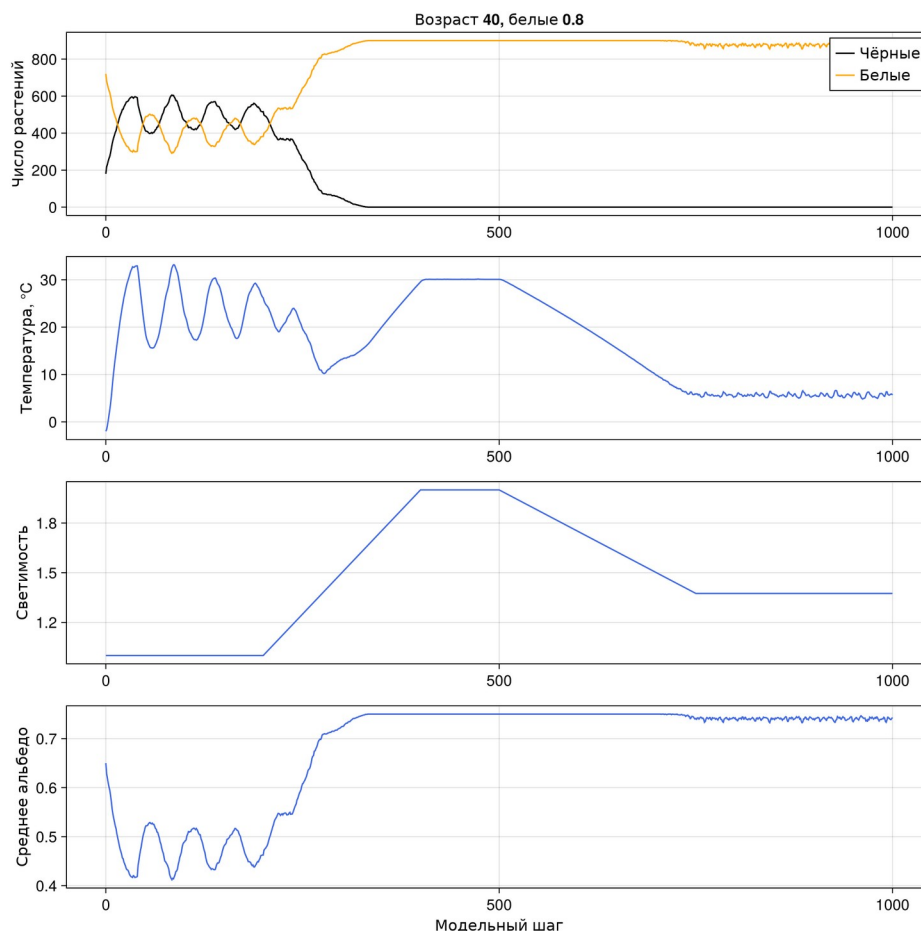


Рисунок 27: Сценарий ramp: $\max_age = 40$, $\text{init_white} = 0.8$

Таблица 7: Конечные состояния параметрической серии ramp

Возраст	Доля белых	Чёрные	Белые	T, °C	Альbedo
25	0,2	0	892	3,938	0,747
25	0,8	0	864	6,258	0,736
40	0,2	0	892	4,115	0,747
40	0,8	0	882	5,667	0,743

В каждом из четырёх сочетаний чёрный вид отсутствует к концу сценария ramp. Белая популяция составляет от 864 до 892 растений. Таким образом, качественный итог воздействия совпадает для выбранной сетки, хотя переходные состояния, численности и температуры различаются. Это вывод для указанных четырёх опытов и seed, а не для всей области параметров.

6. Литературное программирование и Jupyter

Семь файлов в корне `scripts/` — литературные первоисточники. `tangle.jl` получает из каждого чистый Julia-скрипт, выполняемый блокнот IPYNB и Quarto QMD.

Дополнительный статический QMD содержит тот же код в обычных блоках `julia` и включается в тематические разделы настоящего отчёта. Сборка отчёта не повторяет расчёты и не меняет научные результаты.

```
using DrWatson
@quickactivate "project"
using Literate

execute_notebooks = "--execute" in ARGS
sources = filter(!("--execute"), ARGS)
for source in sources
    name = splitext(basename(source))[1]
    Literate.script(source, scriptsdir(name); name, credit = false)
    Literate.notebook(
        source,
        projectdir("notebooks", name);
        name,
        execute = execute_notebooks,
        credit = false,
    )
    Literate.markdown(
        source,
        projectdir("markdown", name);
        name,
        flavor = Literate.QuartoFlavor(),
        credit = false,
    )
    # A static derivative is included in the report without repeating
    # calculations.
    Literate.markdown(
        source,
        projectdir("markdown", name);
        name = name*"-report",
        flavor = Literate.QuartoFlavor(),
        credit = false,
        postprocess = text -> replace(
            replace(text, "`{julia}" => "`julia"),
            r"^(#{1,4}) "m => heading -> "##"*heading,
        ),
    )
end

make notebooks
make docs
jupyter lab --ip=0.0.0.0 --port=8888 --no-browser
```

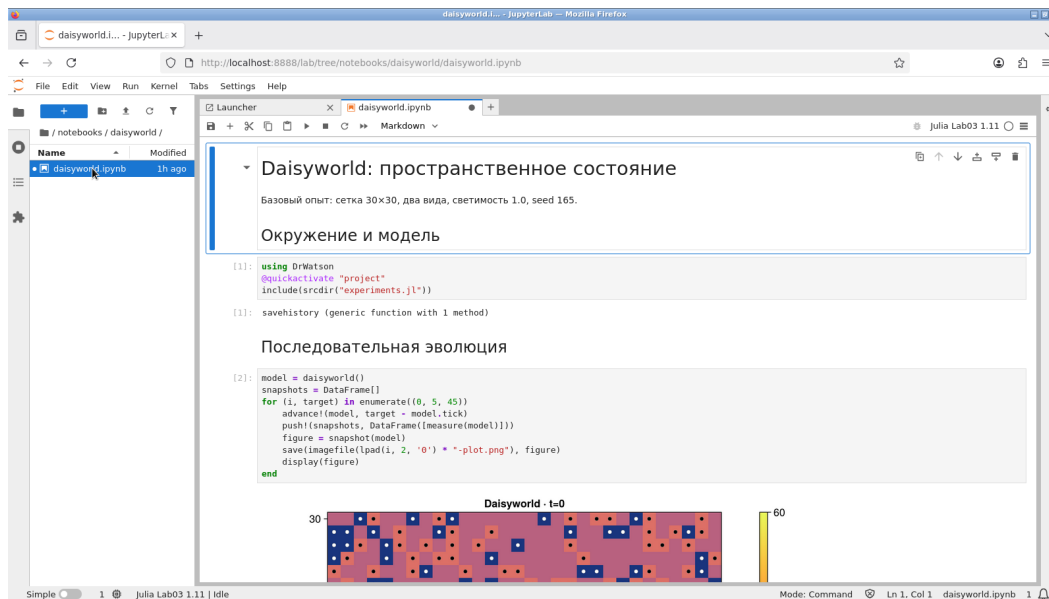


Рисунок 28: Открытый выполненный блокнот Julia

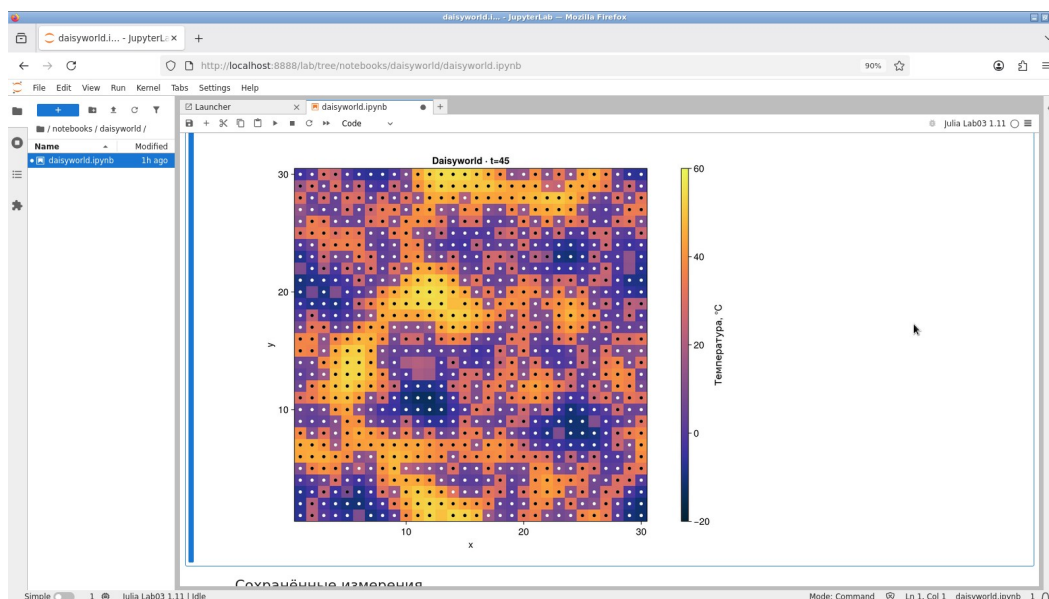


Рисунок 29: Результаты эксперимента в Jupyter

Таблица 8: Проверка выполненных блокнотов

Блокнот	Ячеек	Готово	Ошибок
daisyworld	3	3	0
daisyworld-animate	4	4	0
daisyworld-count	3	3	0
daisyworld-count__param	3	3	0
daisyworld-luminosity	3	3	0
daisyworld-luminosity__param	3	3	0
daisyworld__param	3	3	0

7. Проверки и воспроизведение

Проверены число агентов после инициализации, допустимость долей, предел вместимости решётки, корректность возрастов и температур, воспроизводимость при одинаковом seed и точные границы интервалов изменения светимости. Тесты не являются доказательством климатической достоверности модели: они контролируют реализацию принятого алгоритма.

```
using Test, Agents
include("../src/daisyworld.jl")
using .DaisyworldModel

@testset "Daisyworld: initial state and constraints" begin
    m = daisyworld()
    @test measure(m).white == 180
    @test measure(m).black == 180
    @test length(unique(a.pos for a in allagents(m))) == nagents(m)
    @test all(isfinite, m.temperature)
    @test_throws ArgumentError daisyworld(
        init_white = 0.9,
        init_black = 0.2,
    )
    @test_throws ArgumentError daisyworld(max_age = 0)
    @test_throws ArgumentError daisyworld(griddims = (2, 2))
end

@testset "Daisyworld: dynamics and reproducibility" begin
    a, b = daisyworld(seed = 165), daisyworld(seed = 165)
    advance!(a, 45)
    advance!(b, 45)
    @test measure(a) == measure(b)
    @test a.temperature == b.temperature
    @test a.tick == 45
    @test nagents(a) <= 900
    @test 0 <= measure(a).albedo <= 1
    @test all(x -> x.age < a.max_age, allagents(a))
    empty = daisyworld(init_white = 0.0, init_black = 0.0)
    advance!(empty, 10)
    @test nagents(empty) == 0
end

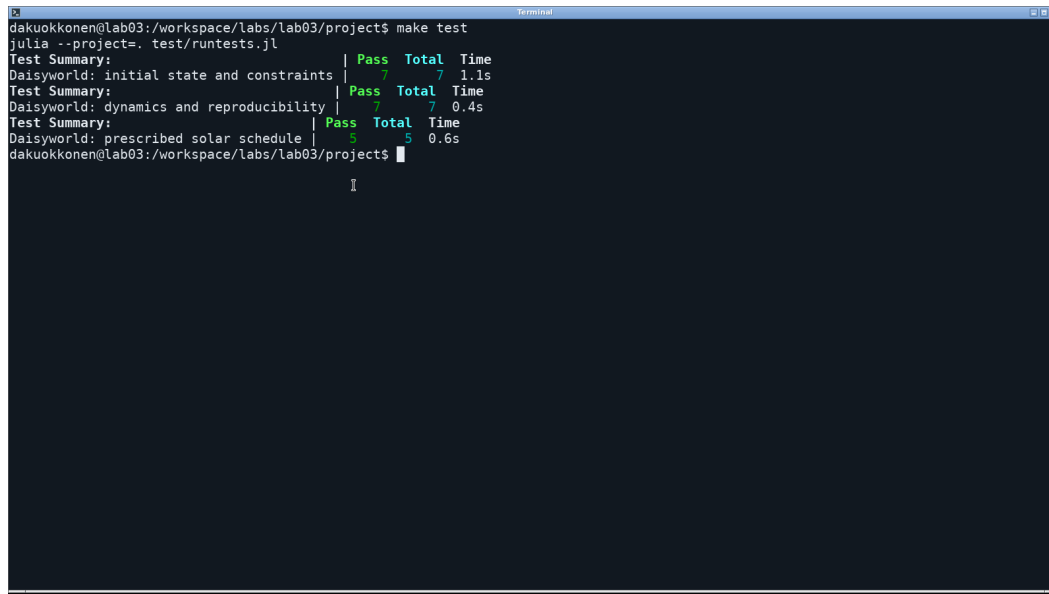
@testset "Daisyworld: prescribed solar schedule" begin
    m = daisyworld(scenario = :ramp, init_white = 0.0, init_black = 0.0)
    advance!(m, 200)
    @test m.solar_luminosity ≈ 1.0
```

```

advance!(m, 200)
@test m.solar_luminosity ≈ 2.0
advance!(m, 100)
@test m.solar_luminosity ≈ 2.0
advance!(m, 250)
@test m.solar_luminosity ≈ 1.375
advance!(m, 250)
@test m.solar_luminosity ≈ 1.375
end

```

Повторный запуск завершился успешно: 19 проверок.



```

dakuokkonen@lab03:/workspace/labs/lab03/project$ make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
Daisyworld: initial state and constraints | 7 7 1.1s
Test Summary: | Pass Total Time
Daisyworld: dynamics and reproducibility | 7 7 0.4s
Test Summary: | Pass Total Time
Daisyworld: prescribed solar schedule | 5 5 0.6s
dakuokkonen@lab03:/workspace/labs/lab03/project$

```

Рисунок 30: Фактический вывод команды `make test`

Из каталога проекта расчёты, блокноты, HTML-документация и тесты запускаются командой `make all`. Сборка отчёта выполняется отдельно:

```

cd ../report
make all

```

Начальный seed обеспечивает повторяемость модели при закреплённых версиях пакетов, порядке обхода и параметрах. Исследование содержит по одному seed на сочетание параметров, поэтому выводы относятся к этим траекториям; доверительные интервалы ансамбля независимых миров не оценивались.

8. Выводы

Реализованы и выполнены семь самостоятельных сценариев Daisyworld, включая анимацию и три параметрических расширения. При постоянной светимости $L = 1$ оба вида сосуществуют с колеблющимися численностями. В базовом сценарии `gamr` чёрный вид исчезает на шаге 310; снижение светимости не возвращает его без механизма повторного заселения. Во всех четырёх параметрических опытах `gamr` конечная популяция состоит только из белых маргариток.

Подготовлены полные временные ряды, контрольные состояния, выполненные блокноты и документация. Проверки подтверждают корректность и повторяемость

алгоритма, но не снимают ограничений учебной модели: решётка мала, обновление асинхронное, мутаций нет, ансамблевая неопределённость не оценивалась.

9. Источники

1. Королькова А. В., Кулябов Д. С. *Имитационное моделирование. Практикум.* Глава 3 «Модель Daisyworld», стр. 115–135.
2. Agents.jl. Документация библиотеки агентного моделирования:
<https://juliadynamics.github.io/Agents.jl/stable/>.
3. DrWatson.jl. Организация научных проектов:
<https://juliadynamics.github.io/DrWatson.jl/stable/>.
4. Literate.jl. Производные форматы литературных исходников:
<https://fredrikekre.github.io/Literate.jl/v2/>.
5. CairoMakie. Построение графиков и запись анимации:
<https://docs.makie.org/stable/>.